CAM-LDS সেশন — বিস্তারিত ব্যাখ্যা

TEMARL থিসিস · আসল CAM-LDS ডেটা ব্যবহারের পুরো প্রক্রিয়া · ২৪ অক্টোবর ২০২৬

> **এই সেশনের এক লাইনের সারাংশ:** আমরা আবিষ্কার করলাম যে এতদিন আমরা CAM-LDS *ডেটাসেট* ব্যবহার করিনি — ব্যবহার করেছি তার *স্ক্রিপ্ট*। তারপর আসল ডেটা নামিয়ে, যাচাই করে, একটা নির্ভুল corpus বানালাম। এই প্রক্রিয়ায় আমার নিজের **চারটি দাবি ভুল প্রমাণিত** হয়েছে।

১. শুরুর প্রশ্নটা কী ছিল

আপনি বললেন — **"হাতে বানানো ডেটা দিয়ে publish করা যাবে না।"**

এই কথাটা থেকেই পুরো সেশন শুরু। প্রশ্নটা ঠিক ছিল, কিন্তু ভেতরে দুটো ভুল ধারণা ছিল যেগুলো পরিষ্কার করা দরকার ছিল:

ধারণা	আসল অবস্থা
"আমাদের ডেটা হাতে বানানো"	**না।** আমাদের technique sequence গুলো **মেশিন দিয়ে parse করা** — AttackBed-এর প্রকাশিত playbook থেকে। শুধু zone/goal অংশটুকু (R4–R6) হাতে টাইপ করা, যেটা মোট ২০ লাইনের মতো JSON।
"CAM-LDS হলো আসল ডেটা"	**পুরোপুরি না।** CAM-LDS নিজেও **simulated testbed**-এর synthetic log। AIT-এর গবেষকরা attack script লিখে সেগুলো চালিয়েছেন। এটা real-world attacker data নয়।

আর publish-এর আসল শর্ত "real data" নয়। নিচের সবগুলো পেপারই simulated data দিয়ে ভালো journal-এ প্রকাশিত:

পেপার	Journal	ডেটা
SG-MAPPO	IEEE IoT-J (Q1)	শুধু simulator
DRD-PPO	Scientific Reports	শুধু simulator
Co-ADAM	Scientific Reports	simulator + public benchmark
Hawkeyes	Computer Networks	নিজের বানানো topology

আসল শর্ত হলো: ডেটা প্রকাশিত হতে হবে, cite করা যেতে হবে, আর আপনার প্রক্রিয়া পুনরাবৃত্তিযোগ্য হতে হবে।

২. প্রথম বড় আবিষ্কার — playbook আর log এক জিনিস নয়

এটাই সেশনের সবচেয়ে গুরুত্বপূর্ণ ধারণাগত পরিষ্কারীকরণ।

জিনিস	কী	আমরা ব্যবহার করেছি?
AttackBed playbook `scenario_1.j2`	আক্রমণের **স্ক্রিপ্ট** — attacker কী কী করবে তার নির্দেশনা। মানুষ লিখেছে, তাই এতে **উদ্দেশ্য ঘোষিত** আছে।	**হ্যাঁ — এটাই আমাদের corpus**
CAM-LDS `Zenodo, 7.41 GB`	ওই স্ক্রিপ্ট **চালানোর ফলে তৈরি log** — apache, syslog, audit, DNS, Suricata, pcap।	**না — কখনো নামাইনি**

> **এখানে একটা সুন্দর যুক্তি লুকিয়ে আছে।** আমাদের corpus R4–R6 (entry zone, target zone, goal) পূরণ করে **কারণ playbook মানুষ লিখেছে** — আর মানুষ লেখার সময় উদ্দেশ্য ঘোষণা করে। CAM-LDS-এর log R4–R6 পূরণ *করতে পারত না*, কারণ log-এর কোনো লেখক নেই যে বলে দেবে attacker কোথায় যেতে চাইছিল।
>
> অর্থাৎ playbook বেছে নেওয়াটা **সঠিক ছিল** — আমরা শুধু ভুল নামে ডাকছিলাম।

থিসিসে যা লিখতে হবে: "attack sequence গুলো AttackBed playbook থেকে সরাসরি parse করা, যে testbed থেকে CAM-LDS সংগ্রহ করা হয়েছে। আমরা released log-এর বদলে playbook ব্যবহার করেছি কারণ আমাদের environment-এর প্রতিটি campaign-এর ঘোষিত entry zone, target zone ও goal দরকার — যেগুলো playbook-এ লেখা আছে, log-এ নেই।"

৩. আরেকটা সংশোধন — CAM-LDS আসলে journal paper

আমি আগে বলেছিলাম CAM-LDS শুধু arXiv preprint। **ওটা ভুল ছিল।**

পুরো নাম	CAM-LDS: cyber attack manifestations for automatic interpretation of system logs and security alerts
লেখক	Landauer, Hotwagner, Boenke, Skopik, Wurzenberger
Journal	**International Journal of Information Security**, vol. 25, 2026
DOI	`10.1007/s10207-026-01318-x`
Quartile	**Q1** (২০২৫, নিশ্চিত) — Q-Cowrie-এর একই journal

তাই আপনার ডেটার উৎস একটা **Q1 journal paper**, preprint নয়। arXiv version নয়, journal version cite করবেন।

৪. কী নামানো হলো

জিনিস	আকার	কাজে লাগল?
`attack_times.csv` (GitHub)	১০৭ KB	**হ্যাঁ — এটাই label-এর উৎস**
`manifestations_filtered.zip`	২১৩.৭ MB	**zone যাচাইয়ের চেষ্টায়**
৩৪টি `scenario_*.zip`	৭.৪২ GB	**audit-এর জন্য**
Packet captures	১৮২ GB	**নামানো হয়নি — দরকার নেই**

> **মনে রাখুন:** আসল কাজের ফাইল মাত্র **১০৭ KB**। ৭.৪২ GB নামানো বৃথা যায়নি — কিন্তু সেটা যাচাই ও audit-এর জন্য, label-এর জন্য নয়।

৫. Audit — ছয়টি প্রশ্ন

#	প্রশ্ন	উত্তর
Q1	কী নামানো হয়েছে?	৩৭টি archive, ৭.৪২ GB, সম্পূর্ণ logs release ✓
Q2	`attackmate.json` আছে?	হ্যাঁ — প্রতি scenario-তে ✓
Q3	ফোল্ডার কাঠামো কী?	`manifestations_filtered/steps/<scenario>-<step>/<host>/`
Q4	Zone বের করা যায়?	**না — ৭টির মধ্যে ২টি মিলল**
Q5	Label আমাদের corpus-এর সাথে মেলে?	মোট সংখ্যা মেলে **(কিন্তু এটা যথেষ্ট নয় — §৬ দেখুন)**

Q6	`attackmate.json`-এ কী আছে?	**কোনো label নেই, কোনো status নেই**

Q6 কেন গুরুত্বপূর্ণ

আমি আগে বলেছিলাম *"attackmate.json ব্যবহার করুন, কারণ এটা আসলে কী চলেছে তা রেকর্ড করে।"* **ভুল ছিল।**

ফাইলটিতে মাত্র চারটি key আছে: `start-datetime`, `type`, `cmd`, `parameters`।

- **কোনো technique label নেই।**
- **কোনো সফল/ব্যর্থ তথ্য নেই।** `parameters`-এর ভেতরের `exit_on_error`, `error_if` ইত্যাদি হলো **playbook-এর সেটিং** — চালানোর *আগে* লেখা, ফলাফল নয়।
- ৩৪টি run, ১,৮০৬টি record — সবগুলোতেই একই।

তাই `attack_times.csv`-ই label-এর একমাত্র উৎস।

৬. Verification — আমার চারটি দাবি ভুল প্রমাণিত

এই অংশটা সবচেয়ে শিক্ষণীয়। বাইরের একটা critique এসেছিল যে আমার কিছু সিদ্ধান্ত "too strong"। যাচাই করে দেখা গেল **critique ঠিক ছিল**।

ভুল ১ — "আমাদের corpus ২% এর মধ্যে মিলে যায়"

আমি **মোট সংখ্যা** মিলিয়ে বলেছিলাম corpus মিলে যায়:

	Release	আমাদের
Step	৯০৪	৮৮৯
Technique instance	১,৩৫১	১,৩৪৭

কিন্তু **মোট সংখ্যা মিললেই sequence মেলে না।** দুটো corpus-এর মোট একই হয়ে ক্রম সম্পূর্ণ ভিন্ন হতে পারে। প্রতিটি run আলাদা করে মিলিয়ে দেখলে:

	সংখ্যা	মানে
হুবহু একই ক্রম	**৫ / ৩৪**	পুরোপুরি মিলে যায়
একই technique, ভিন্ন ক্রম	২৩	set এক, order আলাদা
সত্যিই ভিন্ন	৬	J ০.৯০–০.৯৫

> **এটা কেন বিশেষভাবে গুরুত্বপূর্ণ:** আপনার থিসিসের মূল প্রশ্নই হলো **"ক্রম (order) কি গুরুত্বপূর্ণ?"** — আর আমাদের corpus-এর ক্রমই প্রকাশিত ground truth থেকে আলাদা। এটা ছোটখাটো পার্থক্য নয়।

পাশাপাশি **৩৬ বনাম ৩৪** রহস্যও সমাধান হলো: আমাদের দুটো অতিরিক্ত playbook হলো `scenario_3_a_b.j2` ও `scenario_3_b_b.j2`।

ভুল ২ — "zone বের করা structurally অসম্ভব"

আমি বলেছিলাম এটা "fixable নয়"। কিন্তু আমি আসলে শুধু **দুটো পদ্ধতি** ব্যর্থ হতে দেখেছিলাম:

1. **কোন host-এর log আছে** দেখে — ৭টির মধ্যে ২টি মিলল। কারণ `corpdns` প্রায় সব run-এ প্রথমে আসে (step 1 হলো DNS lookup), আর `inetfw` আসে কারণ traffic firewall পেরোয়। কেউই victim নয়।
2. **command text-এ hostname** খুঁজে — ৯০৪টির মধ্যে মাত্র ১২১টি (১৩.৪%) কোনো host-এর নাম বলে, আর শেষেরটা প্রায় সবসময় `192.42.1.174` — **attacker-এর নিজের IP**।

পরে log-এর **আয়তন** দেখে ৫–৬টি scenario মিলল। কিন্তু এটাও চূড়ান্ত প্রমাণ নয় — একটা web server `nikto/ffuf` scan খেলে হাজার হাজার log লিখবে, সে target হোক বা না হোক।

> **চূড়ান্ত সিদ্ধান্ত:** zone গুলো **হাতে টাইপ করাই থাকবে**, কিন্তু এখন সেটা একটা *প্রমাণভিত্তিক সিদ্ধান্ত* — দুটো পদ্ধতি চেষ্টা করা হয়েছে, দুটোই ব্যর্থ, কেন ব্যর্থ তা চিহ্নিত। এটা limitations section-এ লেখার মতো শক্ত যুক্তি।

ভুল ৩ — "attackmate.json-এ failure/retry রেকর্ড আছে"

§৫-এ ব্যাখ্যা করা। ৩৪টি run-এর সবগুলোতে যাচাই করে দেখা গেছে — নেই।

ভুল ৪ — "variant-level split মানে leakage"

এটা §৮-এ বিস্তারিত। সংক্ষেপে: "leakage" শব্দটা **অতিরঞ্জিত** ছিল।

৭. দুটো সূক্ষ্ম কিন্তু মারাত্মক আবিষ্কার

ক. `event_id` দিয়ে sort করা যাবে না

স্বাভাবিকভাবে মনে হয় `event_id` মানে ধাপের ক্রম। **কিন্তু না।**

যাচাই	ফলাফল
`event_id` ক্রমবর্ধমান নয়	**৩৪-এর মধ্যে ২৭ run**
`event_id` দিয়ে sort করলে সময়ের ক্রম ভেঙে যায়	**৩৪-এর মধ্যে ২৮ run**

উদাহরণ — `2_rootkit` run-এ event_id-এর ক্রম:

২২, ২৩, ২৪, **৩**, ২৫, ২৬, ২৭, ২৮, ২৯, **৫**, ৬, ৭, ৮, …

`event_id` আসলে **playbook-এর ধাপের পরিচয়** (identifier), ক্রম নয়। loop ও retry-র কারণে চালানোর ক্রম আলাদা হয়ে যায়।

> **কেন এটা ভয়ংকর:** যদি `event_id` দিয়ে sort করতাম, তাহলে **২৮টি run-এর sequence ভুল হয়ে যেত** — আর আপনার গবেষণার প্রশ্নই তো ক্রম নিয়ে। এটাই হতো সবচেয়ে বড় নীরব ভুল।
>
> **সমাধান:** শুধু `start` timestamp দিয়ে sort করা হয়।

খ. ৯০৪ বনাম ৮৮৩ — কিছুই হারায়নি

Audit দেখিয়েছিল ৯০৪টি label row কিন্তু ৮৮৩টি folder। মনে হয়েছিল ২১টি ধাপ "হারিয়ে গেছে"।

আসল কারণ: ৯০৪টি row-তে distinct `(scenario, event_id)` জোড়া আছে ৮৮৩টি। ১৮টি key একাধিকবার এসেছে:

Run	Step	বার	Command	সময়ের ব্যবধান
`1_pwnkit_localaccount`	৪০	৩	`rm`	৩২.৭s, ১৬.৪s
`3_vnc_apt`	৫৪	২	`key`	৬০.০s
`6_plugin`	২৬	২	`key`	৫৩.০s

সময়ের ব্যবধান শূন্য নয় — তাই এগুলো **সত্যিকারের পুনরাবৃত্ত execution**, duplicate row নয়।

> **Extractor-এর ফাঁদ:** যদি `(run, event_id)` দিয়ে key করতেন, **২১টি row নীরবে হারিয়ে যেত** — কোনো error ছাড়াই।

৮. সবচেয়ে গুরুত্বপূর্ণ সংখ্যা — split-এর সিদ্ধান্ত

আমি বলেছিলাম "variant-level split মানে leakage"। critique বলল এটা ভুল শব্দ — variant-split *ভুল* নয়, এটা **অন্য প্রশ্নের উত্তর দেয়**। মাপলাম:

একই scenario-র ভেতরে (variant-held-out যা পরীক্ষা করে)

Scenario	Variant	Jaccard গড়	হুবহু একই ক্রম
S1	১৮	**০.৮৫৮**	**০ / ১৫৩**
S3	৬	০.৬৮৭	০ / ১৫
S6	৫	০.৫১৫	০ / ১০

ভিন্ন scenario-র মধ্যে (scenario-held-out যা পরীক্ষা করে)

২১টি জোড়া	Jaccard গড় **০.১১০**	সর্বোচ্চ ০.২৬৭

> **পার্থক্যটা প্রায় ৮ গুণ।** তাই দুটো split **আলাদা প্রশ্নের উত্তর দেয়**:
>
> - **Variant-held-out** → "চেনা campaign-এর নতুন variant-এ কেমন করে?" — বৈধ প্রশ্ন।
> - **Scenario-held-out** → "সম্পূর্ণ অচেনা campaign-এ কেমন করে?" — শুধু এটাই generalisation-এর দাবি সমর্থন করে।
>
> আর গুরুত্বপূর্ণ: S1-এর ১৫৩টি জোড়ার **একটিও হুবহু একই ক্রমের নয়**। তাই variant গুলো duplicate নয় — এজন্যই "leakage" শব্দটা ভুল ছিল।

৯. চূড়ান্ত Extractor — যা তৈরি হলো

পাঁচটি ঘোষিত সিদ্ধান্ত, প্রতিটিই কোনো মাপজোখ দ্বারা বাধ্যতামূলক:

কোড	সিদ্ধান্ত	কারণ
D1	`start` timestamp দিয়ে ক্রম	`event_id` ২৭/৩৪ run-এ অ-ক্রমবর্ধমান
D2	৯০৪টি row-ই রাখা	১৮টি পুনরাবৃত্তির সময় আলাদা — সত্যিকারের execution
D3	Vocabulary নতুন করে তৈরি	পুরনো ৮৪-label চাপিয়ে দেওয়া হবে না
D4	Zone বহন করা, বের করা নয়	দুটো derivation পদ্ধতিই ব্যর্থ
D5	Window/split/collapse নেই	এগুলো modelling সিদ্ধান্ত, পরে

ফলাফল

উৎস row	৯০৪	নির্গত step	৯০৪
বাদ পড়েছে	০	একত্রিত হয়েছে	০
Technique instance	১,৩৫১	Multi-technique step	২০৩ (২২.৫%)
Sub-technique	৯৭	Parent technique	৮১

প্রতিটি record-এ `source_row` আছে — তাই যেকোনো step থেকে `attack_times.csv`-এর নির্দিষ্ট লাইনে ফিরে যাওয়া যায়।

১০. এখন আমরা কোথায় দাঁড়িয়ে

জিনিস	উৎস	অবস্থা
Technique label	`attack_times.csv`	✓ **প্রকাশিত ground truth**
Command text	CSV + attackmate.json	✓
ক্রম	`start` timestamp	✓ **যাচাইকৃত**
Zone / goal (R4–R6)	হাতে টাইপ করা	⚠ **প্রমাণসহ ঘোষিত**
Version	v1, MD5 মিলেছে	✓ **freeze করা**

এখন আপনি সত্যভাবে বলতে পারবেন: "আমরা প্রকাশিত CAM-LDS release-এর ৯০৪-ধাপের labelled ground truth ব্যবহার করেছি" — নিজের reconstruction নয়।

১১. যে সীমাবদ্ধতা দূর হয়নি

> ### ৩৪টি run, ৭টি scenario।
>
> একটি scenario (S1) পুরো corpus-এর **৫৮%**। তিনটি scenario-তে (S4, S5, S7) মাত্র **একটি করে run**।
>
> সঠিকভাবে extract করা **যা মাপছি তার বৈধতা বাড়ায়** — কিন্তু নতুন campaign যোগ করে না। এটা থিসিসে স্পষ্ট করে লিখতে হবে।

১২. পরবর্তী ধাপ

1. **Analysis repo-র commit hash রেকর্ড করুন** — ZIP নামানোয় hash ধরা পড়েনি। `git clone` করে `HEAD` লিখে রাখুন।
2. **S4-এর "Firewall" zone পেপারের topology-র সাথে মিলিয়ে দেখুন** — এটাই একমাত্র অমীমাংসিত zone।
3. **Split protocol freeze করুন** — generalisation দাবির জন্য scenario-held-out; S4/S5/S7 শুধু পুরো হোল্ড-আউট হতে পারে।
4. **প্রতিটি run-এর ভেতরে window বানান** — পুনরাবৃত্তি রেখে দিন (collapse করা আলাদা sensitivity test)।
5. **Loader test ও smoke run** — তারপর পূর্ণ training।

TEMARL · CAM-LDS সেশন · ২৪ অক্টোবর ২০২৬ · সব সংখ্যা ফাইল থেকে যাচাইকৃত
সংশ্লিষ্ট কমিট: `dbde05e` (extractor) · `1fa6beb` (PROVENANCE + similarity audit)